Universidad El Bosque

Facultad Ingeniería

Escuela de sistemas de información territorial



Avance 1 - Proyecto Integrador: Altamira Cine

4 Integrantes:
Jose Espin

Erika Yuliana Gómez Rodríguez

Estefania Paredes

Santiago Rojas

Presentado a:

Francisco Rafael Barguil Vanegas

Bogotá, Colombia

Octubre 15 de 2026

Contenido

Repositorio de Git 3
Identidad de la empresa: Altamira Cine 4
Modelos 7
Modelo Entidad Relación (MER) 7
Modelo Relacional 10
Justificación de la normalización 11
Scripts, capturas y ejecuciones 13
01 - Creación de tablas 13
02 - Datos de prueba 14
03 - Triggers 15
04 - Procedimientos almacenados 19
05 - Consultas 23
Conclusión 26

Repositorio de Git

El repositorio del proyecto se encuentra disponible en el siguiente enlace: https://github.com/tef333/Proyecto-integrador-cinema.

Identidad de la empresa: Altamira Cine



Fig. 1. Logo de Altamira Cine.

El nombre Altamira Cine está inspirado en la cueva de Altamira, ubicada en España, donde se conservan algunas de las primeras representaciones artísticas realizadas por seres humanos. Esta elección representa la relación entre las imágenes y la narración de historias, una forma de expresión que existe desde mucho antes de la aparición del cine.

La programación de Altamira Cine combina estrenos comerciales con producciones del cine colombiano y latinoamericano. Además de las funciones habituales, la empresa organiza eventos especiales que cuentan con la participación de artistas invitados, entre los que se encuentran estrenos con presencia del elenco, conversatorios con directores después de las proyecciones y presentaciones en vivo. De esta manera, la programación contempla tanto la exhibición de películas como actividades que permiten al público interactuar con sus realizadores e intérpretes.

La operación de la empresa se desarrolla de manera presencial en un único complejo de seis salas de cine. La venta de boletas se realiza a través de la taquilla física, atendida por el personal, y de una plataforma en línea que permite a los clientes realizar sus compras de manera autónoma. El pago en efectivo se acepta exclusivamente en la taquilla.

El complejo cuenta con seis salas de diferentes formatos y capacidades, distribuidas de acuerdo con las características de cada espacio. Cinco de ellas se encuentran activas, mientras que la Sala 4DX permanece temporalmente fuera de servicio por mantenimiento. La Tabla 1 presenta la distribución de las salas, sus formatos, su capacidad y su estado operativo.

Sala	Formato	Capacidad (sillas)	Estado
Sala 1	2D	150	Activa
Sala 2	2D	150	Activa
Sala 3	3D	120	Activa
Sala VIP	VIP	60	Activa
Sala IMAX	IMAX	220	Activa
Sala 4DX	4DX	80	En mantenimiento
Total			**780**

Tabla 1. Distribución y capacidad de las salas de Altamira Cine.

La capacidad total del complejo es de 780 sillas, de las cuales 700 corresponden a las cinco salas que se encuentran en operación. La capacidad de cada sala constituye un dato relevante para la gestión de las funciones y la venta de boletas, debido a que el sistema debe impedir que se vendan entradas por encima del aforo disponible.

La empresa cuenta, además, con un registro de 30 actores, directores y artistas provenientes de Colombia y otros países de Iberoamérica. Este registro contempla tanto a las personas que desempeñan un solo rol como a aquellas que cumplen funciones de actuación y dirección. Su información permite gestionar la participación de los artistas en los eventos especiales organizados por el cine.

En cuanto a las particularidades del negocio, Altamira Cine maneja tarifas diferenciadas para el público general, estudiantes, niños, adultos mayores y clientes que adquieren entradas para la sala VIP. El precio aplicado debe conservarse en el registro de cada venta, de manera que las modificaciones posteriores de las tarifas no alteren los valores de las transacciones realizadas anteriormente.

La empresa también dispone de un programa de fidelización que permite acumular puntos asociados al registro de cada espectador. Asimismo, cada película cuenta con una clasificación por edad, establecida en las categorías de todo público, mayores de 7 años, mayores de 12 años, mayores de 15 años y mayores de 18 años. Esta clasificación debe tenerse en cuenta durante el proceso de venta para verificar que la boleta corresponda a las condiciones de acceso establecidas para la película.

Por otra parte, la participación de los artistas en los eventos especiales se formaliza mediante contratos vinculados a funciones específicas. Cada contrato es gestionado por un administrador o coordinador del personal y cuenta con un período de vigencia definido, lo que permite llevar el control de los compromisos adquiridos por la empresa.

Finalmente, el sistema de información debe garantizar el control del aforo y la disponibilidad de las funciones. Para ello, no se deben vender más boletas que la capacidad de la sala

asignada, ni permitir ventas para funciones que ya hayan comenzado o que se encuentren canceladas. Estas condiciones forman parte de las reglas de negocio que deben considerarse en el diseño de la base de datos.

Modelos

Modelo Entidad Relación (MER)

El modelo entidad-relación de Altamira Cine está compuesto por las ocho entidades definidas para representar la operación del negocio: Actores_Directores, Contratos, Salas, Funciones, Tipos_Boleta, Ventas, Espectadores y Staff. Estas entidades permiten organizar la información relacionada con la programación de las películas, la venta de boletas y la gestión de los contratos con los artistas.

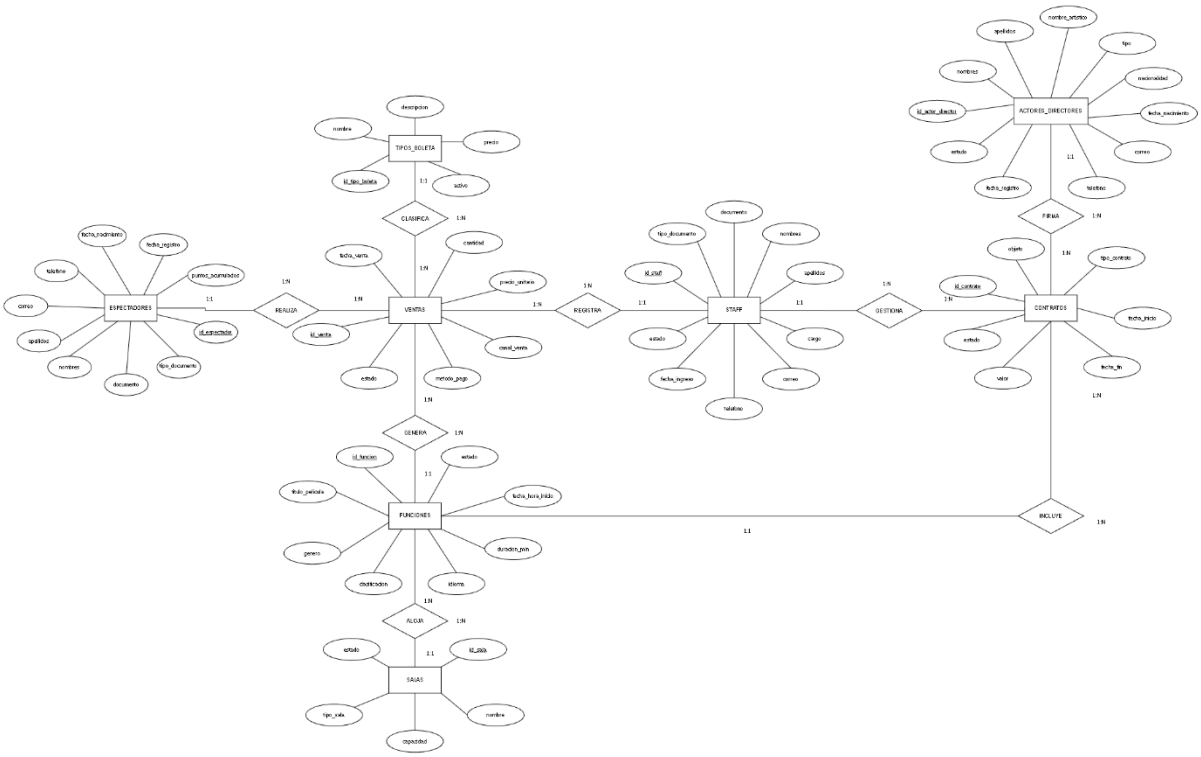


Fig. 2. Modelo entidad-relación).

El diagrama se presenta girado para que los atributos se lean con claridad. El archivo completo, en formato vectorial, está disponible en el repositorio: https://github.com/tef333/Proyecto-integrador-cinema/blob/main/avance1-postgresql/modelo-er/modelo-entidad-relacion.pdf

Para complementar el diagrama, la Tabla 2 presenta las relaciones entre las entidades, sus cardinalidades y la descripción de las reglas que representan dentro del negocio.

Relación	Entidades	Cardinalidad	Explicación
ALOJA	Salas — Funciones	1:N	Una sala aloja muchas funciones; cada función se proyecta en una sola sala.
GENERA	Funciones — Ventas	1:N	Una función genera muchas ventas; cada venta es para una sola función.
REALIZA	Espectadores — Ventas	1:N	Un espectador realiza muchas compras; cada venta pertenece a un solo espectador.
CLASIFICA	Tipos_Boleta — Ventas	1:N	Un tipo de boleta (general, estudiante, VIP…) clasifica muchas ventas.
REGISTRA	Staff — Ventas	1:N (opcional)	Un taquillero registra muchas ventas. En las ventas del canal Online no interviene ningún empleado, por eso esta participación es opcional del lado de la venta.
GESTIONA	Staff — Contratos	1:N	Un administrador o coordinador gestiona muchos contratos; cada contrato lo gestiona un solo empleado.
FIRMA	Actores_Directores — Contratos	1:N	Un artista firma muchos contratos a lo largo del tiempo; cada contrato es con un solo artista.
INCLUYE	Funciones — Contratos	1:N	Una función especial puede incluir varios contratos (varios artistas); cada contrato corresponde a una sola función.

Tabla 2. Tabla explicativa.

Contratos y Ventas son las entidades que concentran las principales transacciones del negocio. La primera relaciona al artista, la función y el empleado encargado de gestionar el contrato, mientras que la segunda registra la compra realizada por el espectador para una función y un tipo de boleta determinados. Cuando la venta se realiza en taquilla, también se registra el empleado que la atiende; en cambio, las ventas en línea no requieren la intervención de un empleado.

Las demás entidades contienen información necesaria para el funcionamiento del sistema, como los datos de los artistas, las salas, las funciones, los tipos de boleta, los espectadores y el personal. En conjunto, las relaciones permiten establecer cómo se vincula esta información dentro del modelo.

Modelo Relacional

A partir del modelo entidad-relación se construyó el modelo relacional, conservando las ocho entidades definidas inicialmente. Las relaciones de muchos a muchos propias del negocio, entre artistas y funciones y entre espectadores y funciones, se resuelven mediante las entidades Contratos y Ventas, que además almacenan atributos propios de cada relación, como el valor y la vigencia del contrato o la cantidad y el precio de la venta. Por esta razón no fue necesario incorporar tablas adicionales.

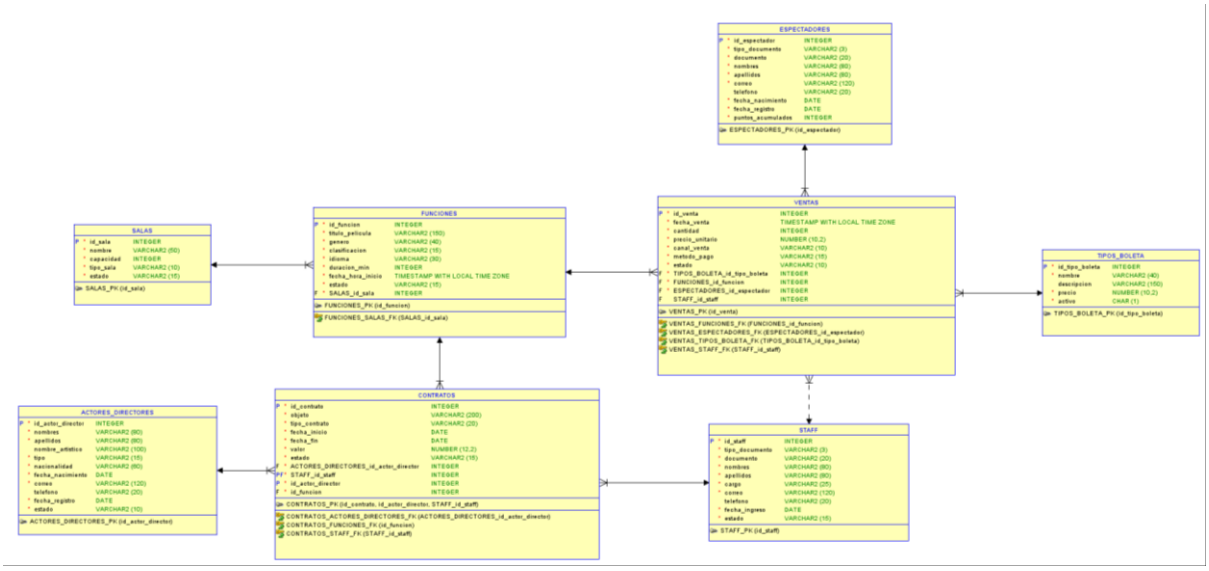


Fig. 3. Modelo relacional.

El modelo relacional se elaboró en Oracle SQL Developer Data Modeler. Por esta razón, los tipos de datos que aparecen en el diagrama corresponden a Oracle y presentan algunas diferencias con respecto a los utilizados posteriormente en el script de creación de tablas para PostgreSQL.

Durante el diseño se estableció que algunos valores se calcularan a partir de otros atributos, en lugar de almacenarse directamente. Este es el caso del total de una venta, obtenido mediante la multiplicación de la cantidad por el precio unitario, y de la hora de finalización de una función, calculada a partir de la hora de inicio y la duración de la película. Estos valores pueden obtenerse mediante consultas, evitando mantener datos derivados que podrían quedar desactualizados si cambian los valores de origen.

En el caso de las ventas, el atributo precio_unitario conserva el precio aplicado al momento de la compra. Aunque este valor proviene del tipo de boleta, debe mantenerse en el registro de la venta para preservar el valor histórico de la transacción. De esta forma, si las tarifas cambian posteriormente, las ventas realizadas con anterioridad mantienen el precio que correspondía en ese momento.

Otra decisión corresponde al alcance definido para el proyecto. El enunciado establece ocho entidades y no contempla una entidad independiente para las películas. Por esta razón, los datos como el título, el género, la clasificación por edad y el idioma se almacenan en la tabla de funciones. Esta decisión simplifica el modelo, aunque implica repetir la información de una película cuando se programa en varias funciones.

El diagrama relacional se generó utilizando los tipos de datos de Oracle SQL Developer Data Modeler. Sin embargo, el script de creación de tablas se implementó en PostgreSQL, por lo que fue necesario adaptar algunos tipos de datos al sistema gestor utilizado.

Entre las correspondencias consideradas se encuentran VARCHAR2 de Oracle con VARCHAR en PostgreSQL, NUMBER con NUMERIC y TIMESTAMP WITH LOCAL TIME ZONE con un tipo de marca temporal compatible con PostgreSQL. Asimismo, los atributos definidos como CHAR(1) pueden representarse mediante BOOLEAN cuando almacenan únicamente valores lógicos. Esta última conversión depende de la finalidad del atributo y de los valores que admite, por lo que no se aplica automáticamente a todos los campos CHAR(1).

Justificación de la normalización

El modelo se verificó contra las tres primeras formas normales. Todas las tablas utilizan como llave primaria un identificador autogenerado, y los demás datos que identifican a una persona o a un objeto del negocio, como el documento, el correo o el nombre de la sala, se protegen con restricciones UNIQUE.

Primera forma normal (1FN). Todos los atributos contienen un único valor atómico y no existen grupos repetidos dentro de una fila. Por ejemplo, los nombres y apellidos están en columnas separadas, cada fila de ventas corresponde a un solo tipo de boleta y cada contrato

a un solo artista y una sola función. Los datos que pueden ser múltiples, como las boletas vendidas para una función o los contratos de un artista, se modelaron como filas de otra tabla y no como listas dentro de una columna.

Segunda forma normal (2FN). Todas las llaves primarias están formadas por una sola columna, por lo que ningún atributo puede depender de solo una parte de la llave. En contratos, la llave primaria es únicamente id_contrato; la combinación de artista y función se protege con una restricción UNIQUE, que impide contratar dos veces al mismo artista para la misma función.

Tercera forma normal (3FN). Ningún atributo que no forma parte de la llave depende de otro atributo no clave. Cada concepto del negocio se almacena en una sola tabla y las demás lo referencian mediante llaves foráneas. Por ejemplo, la capacidad de la sala no se repite en funciones, porque depende de la sala y no de la función; los datos del espectador, como su nombre o su documento, no se copian en ventas, donde solo se guarda id_espectador; y el cargo del empleado no se copia en contratos, sino que se consulta en staff y es validado por un trigger.

Scripts, capturas y ejecuciones

Los scripts de implementación se ejecutaron en orden secuencial sobre una base de datos PostgreSQL limpia, siguiendo la secuencia 01, 02, 03, 04 y 05. El primer script elimina y crea nuevamente las tablas, por lo que permite iniciar la implementación desde cero y repetir el proceso completo de carga. Esta operación debe realizarse teniendo en cuenta que los datos existentes en las tablas eliminadas se pierden.

01 - Creación de tablas

El archivo `01_creacion_tablas.sql` crea las ocho tablas del modelo relacional e incluye las claves primarias y foráneas, las restricciones de integridad, los valores predeterminados y los índices necesarios para la estructura de la base de datos.

Las claves primarias utilizan columnas `IDENTITY` con la opción `GENERATED BY DEFAULT`, lo que permite generar identificadores automáticamente o insertar valores explícitos cuando se requiere coordinar los datos de prueba y sus respectivas claves foráneas. Por su parte, las nueve claves foráneas establecen las relaciones entre las tablas y utilizan las opciones `ON UPDATE CASCADE` y `ON DELETE RESTRICT`. De esta manera, las modificaciones de identificadores relacionados se propagan a los registros dependientes, mientras que se impide eliminar registros que todavía tengan referencias asociadas.

También se definieron restricciones `UNIQUE` para evitar duplicados en los datos que deben ser únicos dentro del negocio, como los documentos de identidad, los correos electrónicos y determinadas combinaciones de sala, horario y contratación de artistas. Las restricciones `CHECK` limitan los valores permitidos para estados, tipos, clasificaciones y canales, además de validar rangos numéricos, precios, formatos de correo electrónico y fechas de vigencia de los contratos. Entre los rangos establecidos se encuentran capacidades de sala entre 1 y 1000 sillas, duraciones entre 1 y 400 minutos y cantidades de entre 1 y 10 boletas por venta.

En la tabla de ventas se incorporaron restricciones adicionales para garantizar la correspondencia entre el canal de venta y el empleado responsable. Las ventas realizadas en línea no deben tener un taquillero asociado, mientras que las ventas efectuadas en taquilla requieren registrar al empleado que las atiende. Asimismo, el pago en efectivo está permitido únicamente para las ventas realizadas en taquilla.

Para completar las reglas de integridad, se establecieron valores predeterminados mediante `DEFAULT` para determinados estados y fechas de registro, así como restricciones `NOT NULL` en los atributos obligatorios. El script también crea 14 índices destinados a facilitar las consultas sobre claves foráneas y columnas de búsqueda frecuente, entre ellas apellidos, cargo, fecha de función, título de película, fecha de venta, estado y fecha de finalización de

contratos. Las claves primarias y las restricciones de unicidad cuentan, además, con sus propios índices.

Finalmente, se incluyeron instrucciones `COMMENT ON TABLE` para documentar cada tabla dentro de la base de datos. Una vez ejecutado el script, se verificó la creación de las ocho tablas mediante la consulta final, que muestra cada tabla junto con su número de columnas.

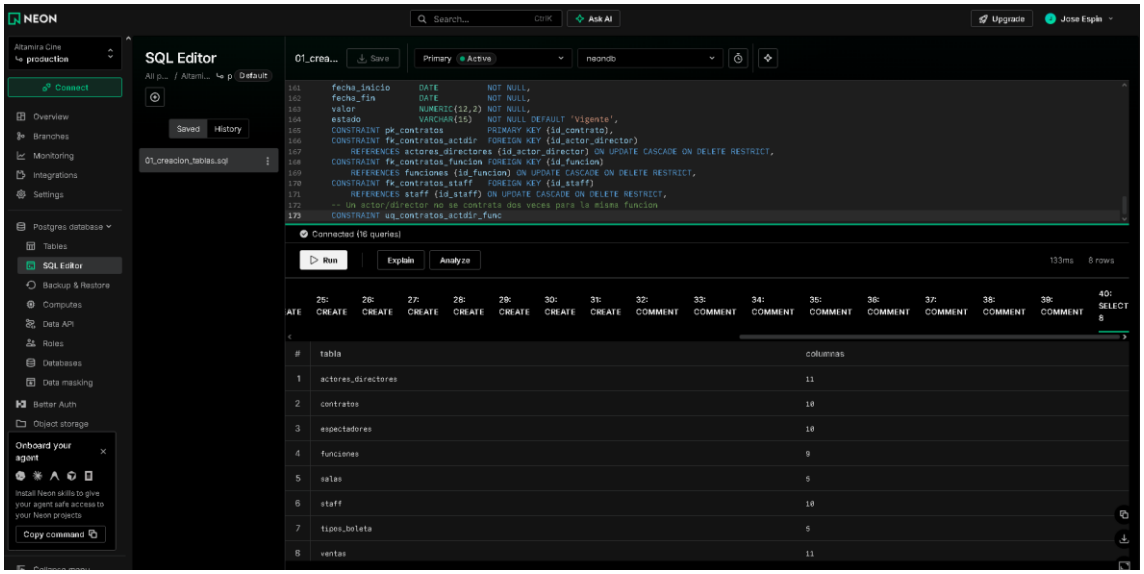


Fig. 4. Resultado de la ejecución del script de creación de tablas

02 - Datos de prueba

El archivo `02_datos_prueba.sql` contiene la carga de los datos de prueba de la base de datos. Cada integrante se encargó de insertar los registros correspondientes a las tablas que tenía asignadas, respetando el orden de las llaves foráneas, tal como se encuentra organizado en el repositorio de GitHub.

Se cumple con el mínimo de 30 registros en las tablas Actores_Directores, Espectadores, Ventas y Funciones. Las demás tablas también cuentan con datos relacionados con estos registros, de manera que la información mantiene coherencia entre las entidades.

Los datos se prepararon para representar diferentes situaciones del funcionamiento de Altamira Cine. Las funciones futuras utilizan `NOW() + INTERVAL` en lugar de fechas fijas, lo que permite probar los triggers y procedimientos en el momento en que se ejecute el script. También se incluyen funciones finalizadas, canceladas y programadas; las ventas asociadas a funciones canceladas se encuentran anuladas.

Asimismo, las ventas respetan las condiciones establecidas para los canales de atención y los medios de pago: las compras en línea no tienen un taquillero asociado y los pagos en efectivo corresponden únicamente a ventas realizadas en taquilla. En cuanto a los contratos, aquellos asociados a funciones ya realizadas figuran como finalizados, los correspondientes a funciones canceladas aparecen como cancelados y los vinculados a funciones futuras se encuentran vigentes. Las fechas de vigencia se calculan a partir de la fecha de cada función.

La carga también contempla situaciones específicas para probar el comportamiento de la base de datos, como una función casi llena, funciones con poca asistencia, un tipo de boleta retirado, una sala en mantenimiento, un empleado de vacaciones y dos artistas inactivos.

Finalmente, el script sincroniza las secuencias de los identificadores mediante `setval`, con el fin de evitar conflictos entre los identificadores insertados explícitamente durante la carga y los que se generen en inserciones posteriores.

Para verificar la cantidad de registros cargados en cada tabla, se utiliza una consulta que cuenta los registros y presenta los resultados mediante `UNION ALL`. Esta consulta permite comprobar el contenido de las ocho tablas después de ejecutar el script.

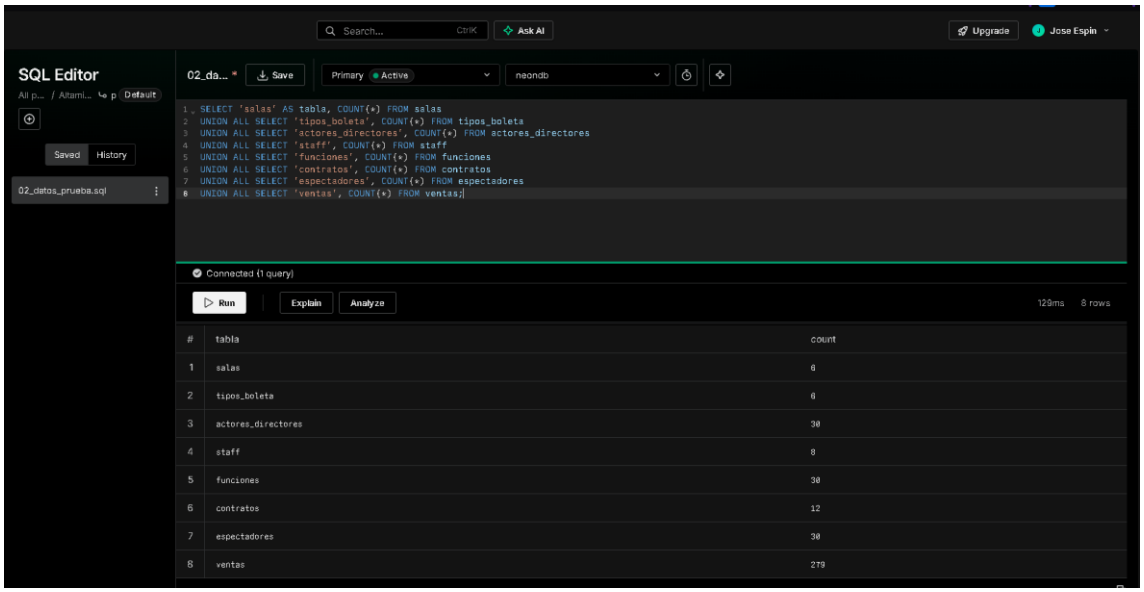


Fig. 5. Ejecución del script de carga de datos de prueba y conteo de registros por tabla.

03 - Triggers

El archivo `03_triggers.sql` implementa ocho triggers distribuidos en seis de las ocho tablas del modelo: Funciones, Ventas, Salas, Contratos, Actores_Directores y Espectadores. Con esta implementación se supera el mínimo establecido de seis triggers distribuidos en seis tablas. Estos mecanismos permiten validar reglas de negocio que requieren consultar otras

filas o tablas, así como normalizar información antes de almacenarla y actualizar datos relacionados después de una operación.

Validación de la programación de funciones.

El trigger `trg_funciones_validar_programacion` se ejecuta antes de insertar o actualizar registros en la tabla Funciones. Su función es comprobar que una función programada se asigne a una sala activa y que su horario no se cruce con el de otra función de la misma sala. Esta validación requiere consultar otras funciones y verificar el estado de la sala, información que no puede comprobarse únicamente mediante una restricción `CHECK`. Además, se utiliza `FOR UPDATE` para bloquear la sala durante la validación y evitar conflictos por programaciones simultáneas.

Validación de las ventas y del aforo.

El trigger `trg_ventas_validar_cupo` se ejecuta antes de insertar o actualizar registros en Ventas. Este comprueba que las boletas correspondan a funciones programadas que todavía no han comenzado, que exista capacidad disponible en la sala y que el tipo de boleta se encuentre activo. Cuando la venta no incluye un precio, se toma el precio vigente del tipo de boleta. Como la disponibilidad depende de la capacidad de la sala y de las ventas existentes para la función, el trigger consulta información de varias tablas. También utiliza `FOR UPDATE` para evitar que dos operaciones simultáneas vendan la última silla disponible.

Validación de la edad del espectador.

El trigger `trg_ventas_validar_edad` se ejecuta antes de insertar o actualizar una venta. Su propósito es verificar que el espectador cumpla la edad mínima exigida por la clasificación de la película para la fecha de la función. Para realizar esta comprobación, consulta la fecha de nacimiento registrada en Espectadores y la clasificación almacenada en Funciones, ya que ambos datos son necesarios para determinar si se cumple la condición de acceso.

Validación de los cambios en las salas.

El trigger `trg_salas_validar_cambios` se ejecuta antes de actualizar el estado o la capacidad de una sala. La validación impide retirar del servicio una sala que tenga funciones programadas a futuro y evita reducir su capacidad por debajo de la cantidad de boletas vendidas para alguna de esas funciones. Estas condiciones requieren consultar las tablas Funciones y Ventas, debido a que dependen de la programación y de las ventas existentes.

Validación de los contratos. El trigger `trg_contratos_validar` se ejecuta antes de insertar o actualizar un contrato. Comprueba que un contrato vigente sea gestionado por un administrador o coordinador activo, que el artista contratado también se encuentre activo y

que la función asociada no esté cancelada. Asimismo, verifica que la fecha de la función esté comprendida dentro del período de vigencia del contrato. Estas condiciones requieren consultar los datos del personal, el estado de los artistas y la información de las funciones relacionadas.

Validación de la inactivación de artistas.

El trigger `trg_actores_validar_inactivacion` se ejecuta antes de actualizar el estado de un registro en Actores_Directores. Su función es impedir que un artista se marque como inactivo mientras tenga contratos vigentes. Para permitir el cambio de estado, primero deben finalizarse o cancelarse los contratos correspondientes. Esta validación requiere consultar la tabla Contratos.

Normalización de los datos de los espectadores.

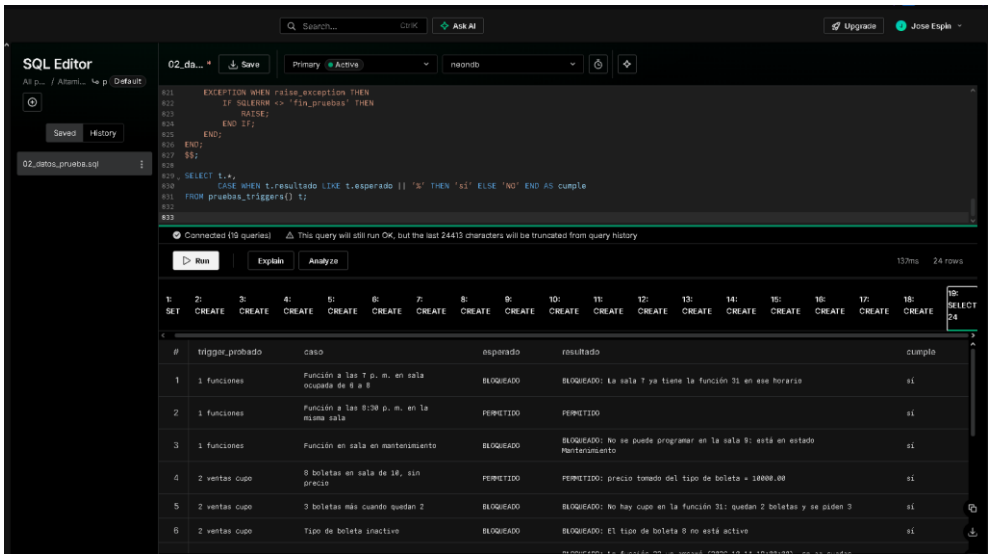
El trigger `trg_espectadores_normalizar` se ejecuta antes de insertar o actualizar registros en Espectadores. Antes de guardar la información, convierte los correos electrónicos a minúsculas y elimina sus espacios, normaliza los documentos de identidad mediante el uso de letras y números en mayúsculas y ajusta los nombres y apellidos para que tengan inicial mayúscula. Esta transformación permite que las restricciones de unicidad detecten registros duplicados aunque la información se haya escrito con diferencias de formato. Por ejemplo, evita que Ana@Mail.com y ana@mail.com se almacenen como correos diferentes, o que un mismo documento se registre con y sin puntos.

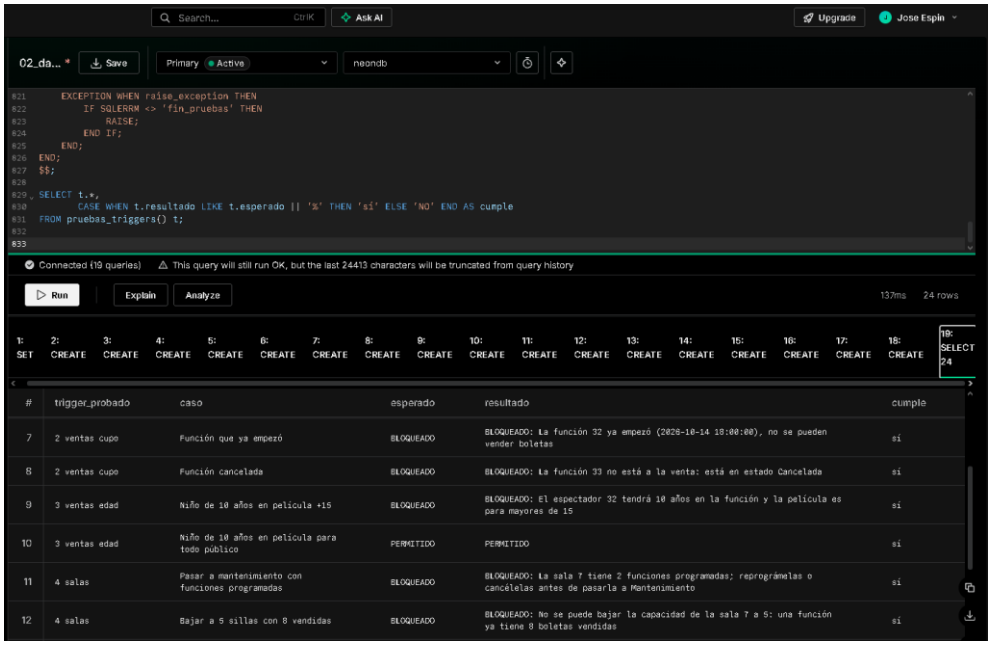
Actualización de los puntos de fidelización.

El trigger `trg_ventas_acumular_puntos` se ejecuta después de insertar, actualizar o eliminar registros en Ventas. Su función es mantener actualizado el saldo de puntos del espectador: cada venta pagada suma un punto por cada $1.000 de su valor. Si una venta se anula, se elimina o cambia su cantidad, precio o espectador asociado, el saldo se ajusta según la operación realizada. Para evitar saldos negativos, se utiliza `GREATEST`, teniendo en cuenta que los puntos cargados mediante el script de datos de prueba se consideran el saldo de apertura. De esta manera, la actualización de los puntos queda asociada a las operaciones sobre las ventas.

Pruebas de funcionamiento.

Al finalizar, el script ejecuta una función de verificación que evalúa los ocho triggers mediante casos que deben ser rechazados y casos que deben ser aceptados, para un total de 24 pruebas. Los resultados se registran para comprobar el comportamiento de cada validación, y las operaciones de prueba se deshacen para evitar que queden datos adicionales en la base de datos. La columna `cumple` permite identificar el resultado de cada caso y debe mostrar el valor «sí» en todas las filas cuando las pruebas se ejecutan correctamente.


SQL Editor
02_datos_pruebas.sql
Saved
History
02_da... *
Save
Primary
Active
neondb
Search...
Ask AI
Upgrade
Jose Espin
Connected (19 queries)
This query will still run OK, but the last 24413 characters will be truncated from query history
Run
Explain
Analyze
137ms
24 rows
1: SET
2: CREATE
3: CREATE
4: CREATE
5: CREATE
6: CREATE
7: CREATE
8: CREATE
9: CREATE
10: CREATE
11: CREATE
12: CREATE
13: CREATE
14: CREATE
15: CREATE
16: CREATE
17: CREATE
18: CREATE
19: SELECT 24
| trigger_probado | caso | esperado | resultado | cumple
1 | 1 funciones | Función a las 7 p. m. en sala ocupada de 8 a 8 | BLOQUEADO | BLOQUEADO: La sala 7 ya tiene la función 31 en ese horario | sí
2 | 1 funciones | Función a las 8:30 p. m. en la misma sala | PERMITIDO | PERMITIDO | sí
3 | 1 funciones | Función en sala en mantenimiento | BLOQUEADO | BLOQUEADO: No se puede programar en la sala 9: está en estado Mantenimiento | sí
4 | 2 ventas cupo | 8 boletas en sala de 10, sin precio | PERMITIDO | PERMITIDO: precio tomado del tipo de boleta = 10000.00 | sí
5 | 2 ventas cupo | 3 boletas más cuando quedan 2 | BLOQUEADO | BLOQUEADO: No hay cupo en la función 31: quedan 2 boletas y se piden 3 | sí
6 | 2 ventas cupo | Tipo de boleta inactive | BLOQUEADO | BLOQUEADO: El tipo de boleta 8 no está active | sí


Search...
Ask AI
Upgrade
Jose Espin
02_da... *
Save
Primary
Active
neondb
Connected (19 queries)
This query will still run OK, but the last 24413 characters will be truncated from query history
Run
Explain
Analyze
137ms
24 rows
1: SET
2: CREATE
3: CREATE
4: CREATE
5: CREATE
6: CREATE
7: CREATE
8: CREATE
9: CREATE
10: CREATE
11: CREATE
12: CREATE
13: CREATE
14: CREATE
15: CREATE
16: CREATE
17: CREATE
18: CREATE
19: SELECT 24
| trigger_probado | caso | esperado | resultado | cumple
7 | 2 ventas cupo | Función que ya empezó | BLOQUEADO | BLOQUEADO: La función 32 ya empezó (2026-10-14 18:00:00), no se pueden vender boletas | sí
8 | 2 ventas cupo | Función cancelada | BLOQUEADO | BLOQUEADO: La función 33 no está a la venta: está en estado Cancelada | sí
9 | 3 ventas edad | Niño de 10 años en película +15 | BLOQUEADO | BLOQUEADO: El espectador 32 tendrá 10 años en la función y la película es para mayores de 15 | sí
10 | 3 ventas edad | Niño de 10 años en película para todo público | PERMITIDO | PERMITIDO | sí
11 | 4 salas | Pasar a mantenimiento con funciones programadas | BLOQUEADO | BLOQUEADO: La sala 7 tiene 2 funciones programadas; reprográmelas o cancélelas antes de pasarla a Mantenimiento | sí
12 | 4 salas | Bajar a 5 sillas con 8 vendidas | BLOQUEADO | BLOQUEADO: No se puede bajar la capacidad de la sala 7 a 5: una función ya tiene 8 boletas vendidas | sí

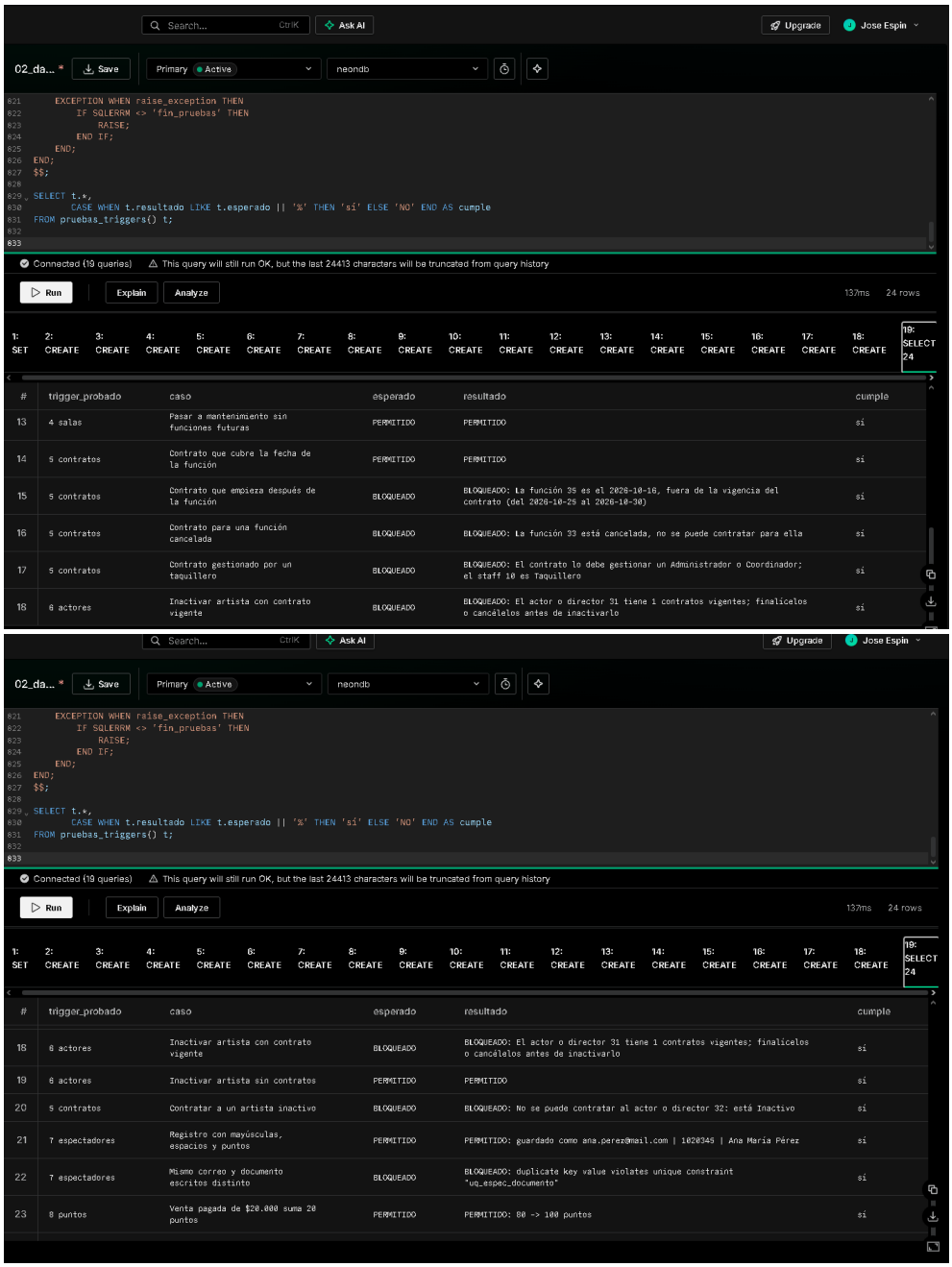


Fig. 6. Resultados de las pruebas de funcionamiento de los triggers.

04 - Procedimientos almacenados

El archivo 04_procedimientos.sql contiene tres procedimientos almacenados, uno por cada categoría requerida: cálculo y manejo de transacciones, generación de reportes y mantenimiento de la base de datos. Los procedimientos incluyen manejo de excepciones mediante EXCEPTION y RAISE, de manera que los errores relacionados con las reglas del negocio se informan con su mensaje original, mientras que los errores inesperados se reportan mediante su código SQLSTATE.

Cancelación de funciones. El procedimiento sp_cancelar_funcion recibe como parámetros el identificador de la función y un motivo opcional. Su ejecución bloquea la función y verifica que exista, que se encuentre programada y que todavía no haya comenzado. Si se cumplen estas condiciones, calcula el valor que debe reembolsarse a partir de la suma de la cantidad por el precio de las ventas pagadas, anula dichas ventas, cancela los contratos vigentes asociados y cambia el estado de la función a Cancelada.

Si la función no existe, no se encuentra programada o ya comenzó, el procedimiento genera el error correspondiente. Además, si ocurre un error durante la operación, los cambios se revierten. La anulación de las ventas activa el trigger encargado de actualizar los puntos de fidelización de los espectadores.

Para comprobar su funcionamiento se canceló la función 22 por una falla técnica del proyector y, después, se intentó cancelar la función 1, que ya está finalizada, para verificar que el procedimiento rechace una operación no permitida.

```
CALL sp_cancelar_funcion(22, 'Falla técnica del proyector');
CALL sp_cancelar_funcion(1, 'Prueba');
```

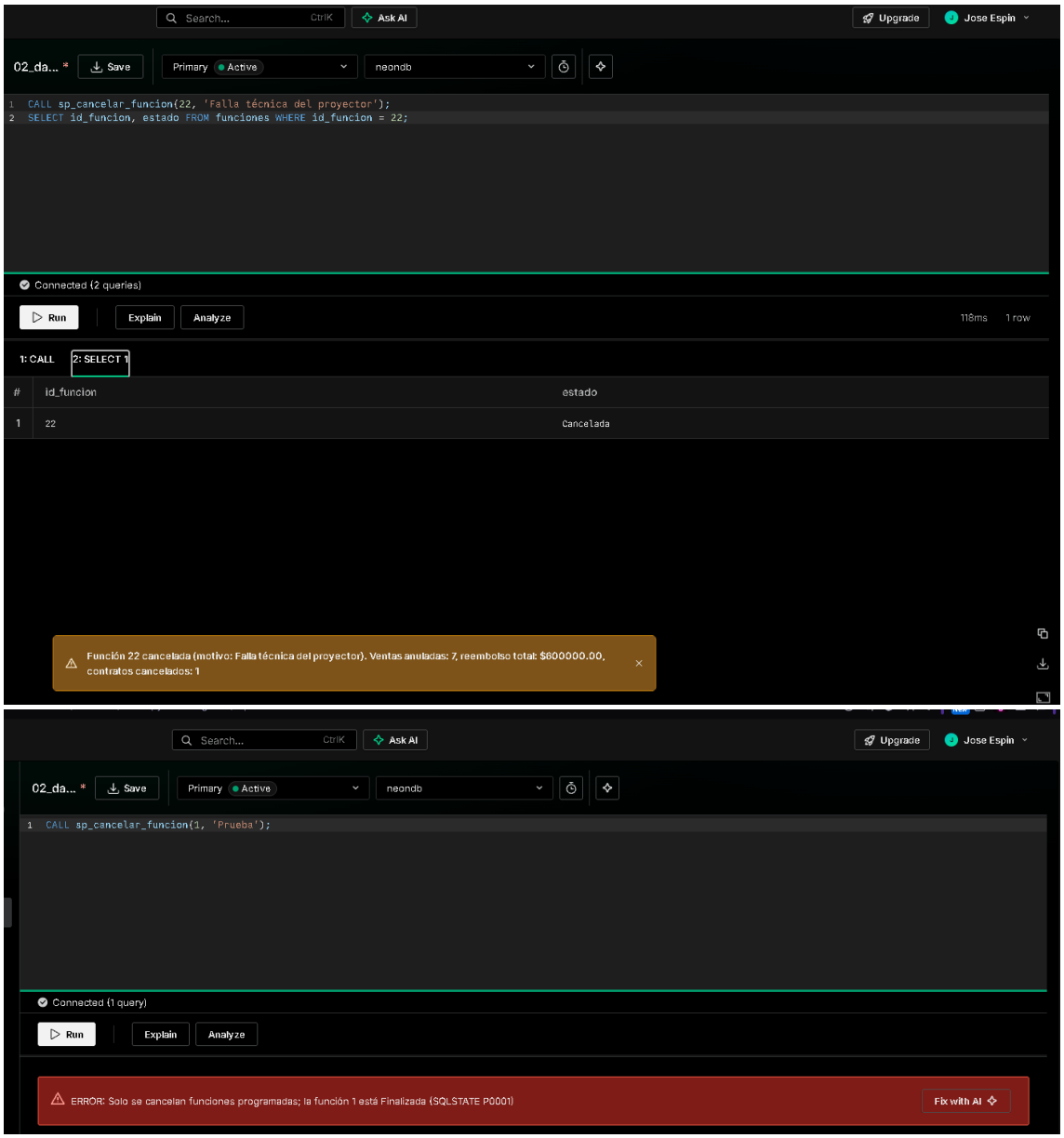


Fig. 7. Ejecución del procedimiento de cancelación de funciones y comprobación de errores.

La Fig. 7 muestra el mensaje de la cancelación, con las 7 ventas anuladas, el reembolso total de $600.000 y el contrato cancelado, así como el error que se genera al intentar cancelar una función finalizada.

Generación del reporte de ventas. El procedimiento sp_reporte_ventas recibe como parámetros las fechas inicial y final del período que se desea consultar, tomando como referencia las fechas de las funciones. El reporte presenta, por película, la cantidad de funciones realizadas dentro del rango, las boletas vendidas, los ingresos y el porcentaje de ocupación calculado a partir de las boletas vendidas y la capacidad de las salas. Los resultados se ordenan por ingresos y se incluye una fila con los totales.

Para obtener estos resultados, el procedimiento utiliza una expresión de tabla común (CTE) que agrupa la información por función y posteriormente la consolida por película. Se excluyen las funciones canceladas y las ventas anuladas. Asimismo, se validan los parámetros recibidos para impedir la ejecución con fechas nulas o con un rango en el que la fecha inicial sea posterior a la final.

La siguiente llamada genera el reporte de las funciones de los últimos 60 días y de los próximos 30:

CALL sp_reporte_ventas(CURRENT_DATE - 60, CURRENT_DATE + 30);

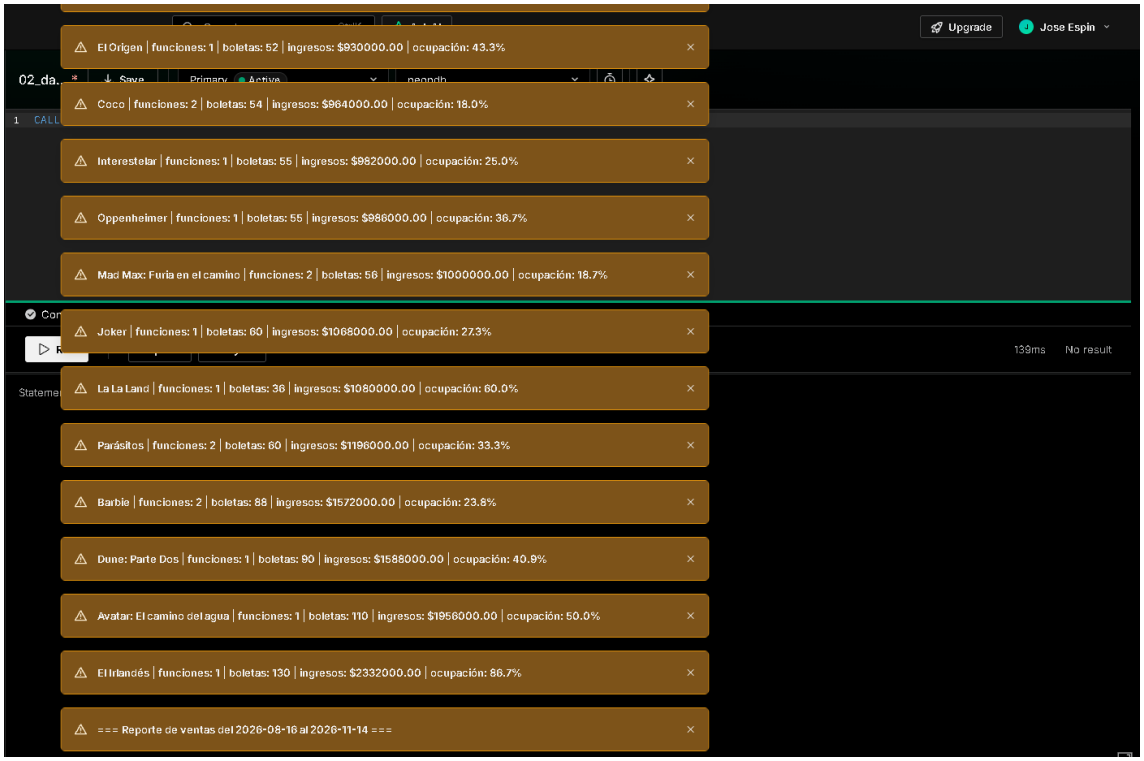


Fig. 8. Resultado del reporte de ventas por película.

Mantenimiento diario. El procedimiento sp_mantenimiento_diario no requiere parámetros y está diseñado para ejecutarse periódicamente. Su función es actualizar el estado de las funciones según sus horarios: las que ya terminaron pasan a Finalizada, mientras que aquellas que comenzaron y aún no han terminado pasan a En curso. También finaliza los contratos vigentes cuya fecha de finalización es anterior a la fecha actual.

Al terminar, el procedimiento informa la cantidad de registros modificados en cada caso. Si se presenta un error inesperado, este se reporta mediante su código SQLSTATE.

La ejecución se realiza mediante la siguiente instrucción:

CALL sp_mantenimiento_diario();

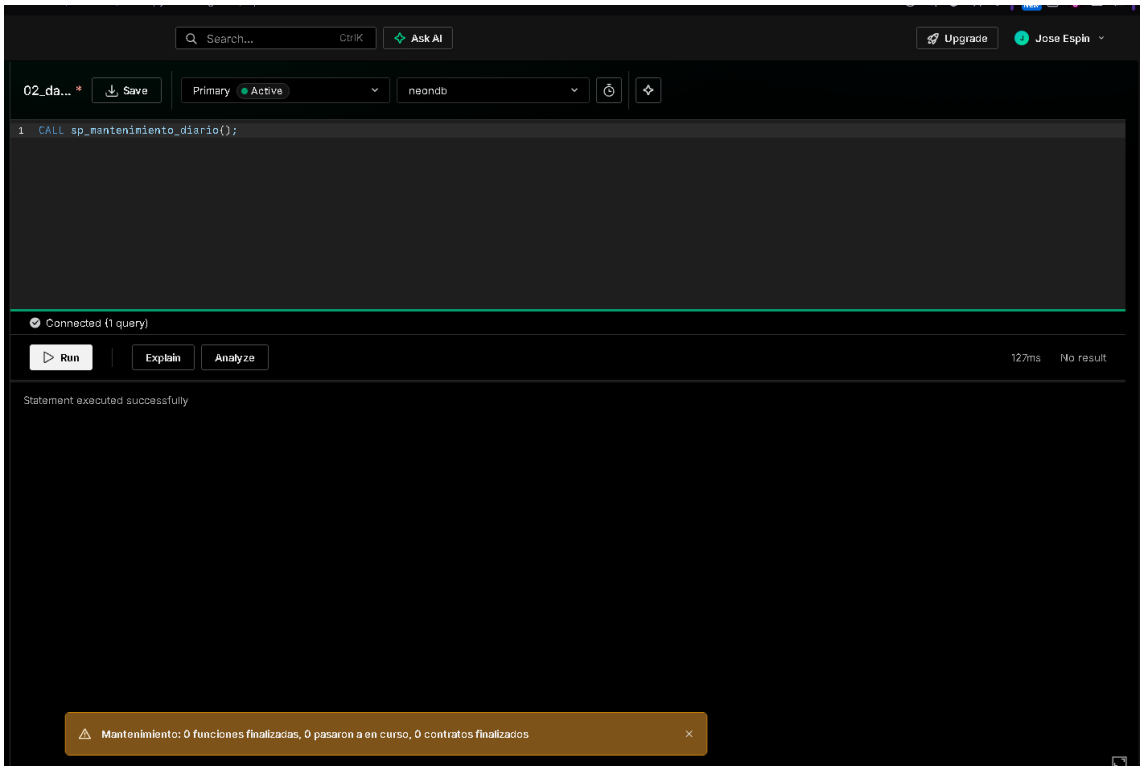


Fig. 9. Resultado de la ejecución del procedimiento de mantenimiento diario.

05 - Consultas

El archivo 05_consultas.sql contiene tres consultas orientadas al análisis de la información de Altamira Cine. Estas utilizan combinaciones de tablas mediante JOIN, funciones de agregación y subconsultas para obtener resultados relacionados con la ocupación de las funciones, el comportamiento de compra de los espectadores y la relación entre los costos de los contratos y los ingresos de taquilla. Las consultas son de lectura y consideran únicamente las ventas pagadas y las funciones no canceladas, según los criterios establecidos para cada análisis.

Funciones con ocupación inferior al promedio. La primera consulta permite identificar las funciones finalizadas cuya ocupación se encuentra por debajo del promedio general. Esta información puede utilizarse para evaluar la programación de determinadas películas u horarios, considerar salas de menor capacidad o analizar la posibilidad de realizar promociones. Se incluyen únicamente funciones finalizadas, debido a que las funciones futuras todavía pueden recibir ventas y presentar una ocupación baja que no refleje su resultado definitivo.

Para obtener los resultados, se relacionan las tablas Funciones y Salas y se utiliza un LEFT JOIN con Ventas, de modo que también se incluyan las funciones que no registran ventas. La ocupación se calcula mediante SUM y GROUP BY, mientras que una CTE y una subconsulta

escalar con AVG permiten obtener el promedio de referencia. Cada fila corresponde a una función cuya ocupación está por debajo de dicho promedio. Los resultados se ordenan desde la función con menor ocupación hasta la de mayor ocupación dentro del conjunto seleccionado.

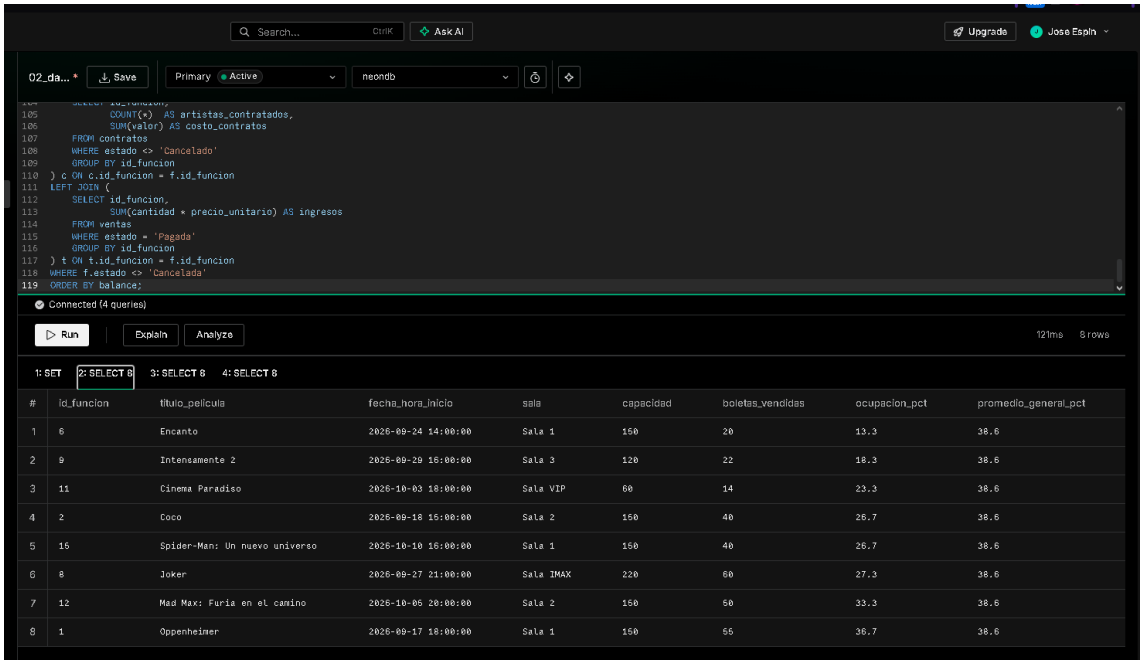


Fig. 10. Funciones finalizadas con ocupación inferior al promedio.

Espectadores cuyo gasto supera el promedio. La segunda consulta identifica a los espectadores cuyo gasto total supera el gasto promedio por espectador. El resultado permite reconocer a los clientes que podrían considerarse en iniciativas de fidelización, como descuentos, preventas o invitaciones.

La consulta relaciona las tablas Ventas y Espectadores y agrupa la información mediante GROUP BY, utilizando las funciones SUM y COUNT para calcular los valores correspondientes. Posteriormente, aplica HAVING con una subconsulta que determina el gasto promedio por espectador. Los resultados presentan el gasto total, el número de compras, la cantidad de boletas y los puntos acumulados de cada cliente, ordenados de mayor a menor gasto.

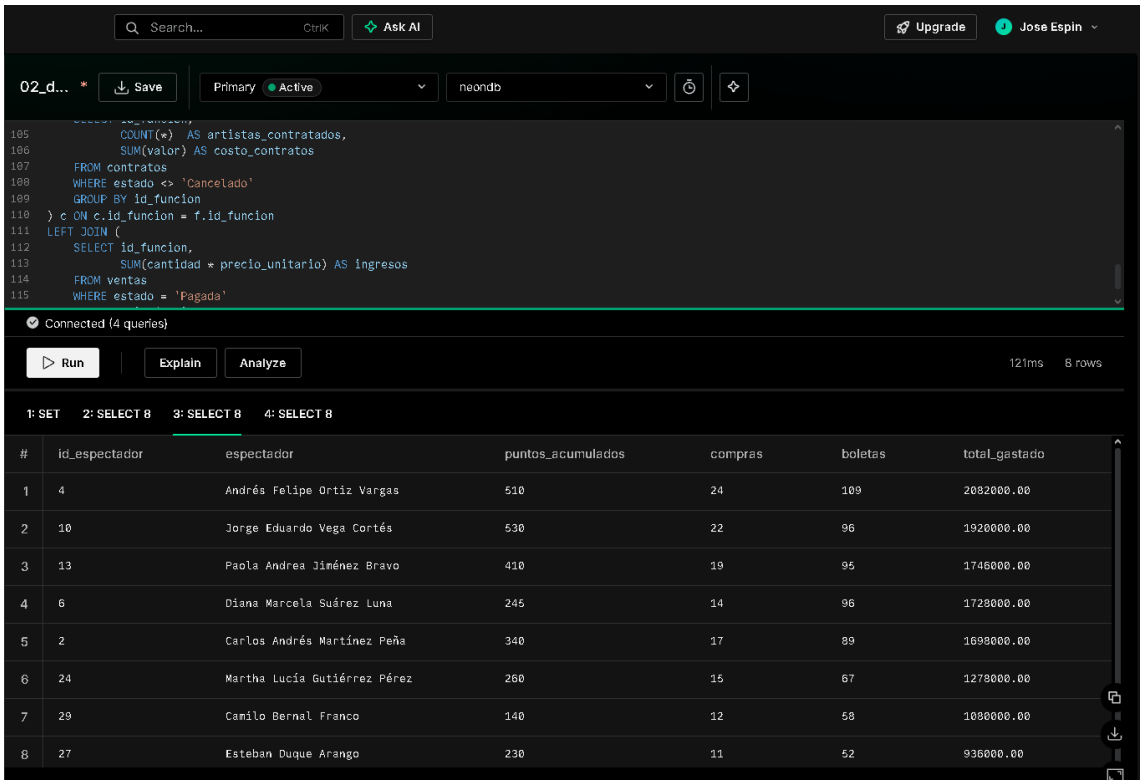


Fig. 11. Espectadores con gasto total superior al promedio.

Comparación entre el costo de los contratos y los ingresos de taquilla. La tercera consulta permite comparar el costo de los contratos asociados a una función con los ingresos obtenidos por la venta de boletas. Su propósito es evaluar si la taquilla cubre los costos de contratación de los artistas que participan en funciones especiales.

Para evitar que los valores se multipliquen por la combinación de varias filas durante los cruces entre tablas, se utilizan subconsultas en la cláusula FROM que agrupan por separado los contratos y las ventas. A partir de estos resultados se calcula el balance entre los ingresos de taquilla y los costos de los contratos. Un balance negativo indica que los ingresos registrados no alcanzan a cubrir dichos costos.

En las funciones que todavía se encuentran programadas, el balance es parcial, puesto que la venta de boletas continúa abierta. Por esta razón, también se presenta el estado de la función, que permite interpretar los resultados teniendo en cuenta su situación actual.

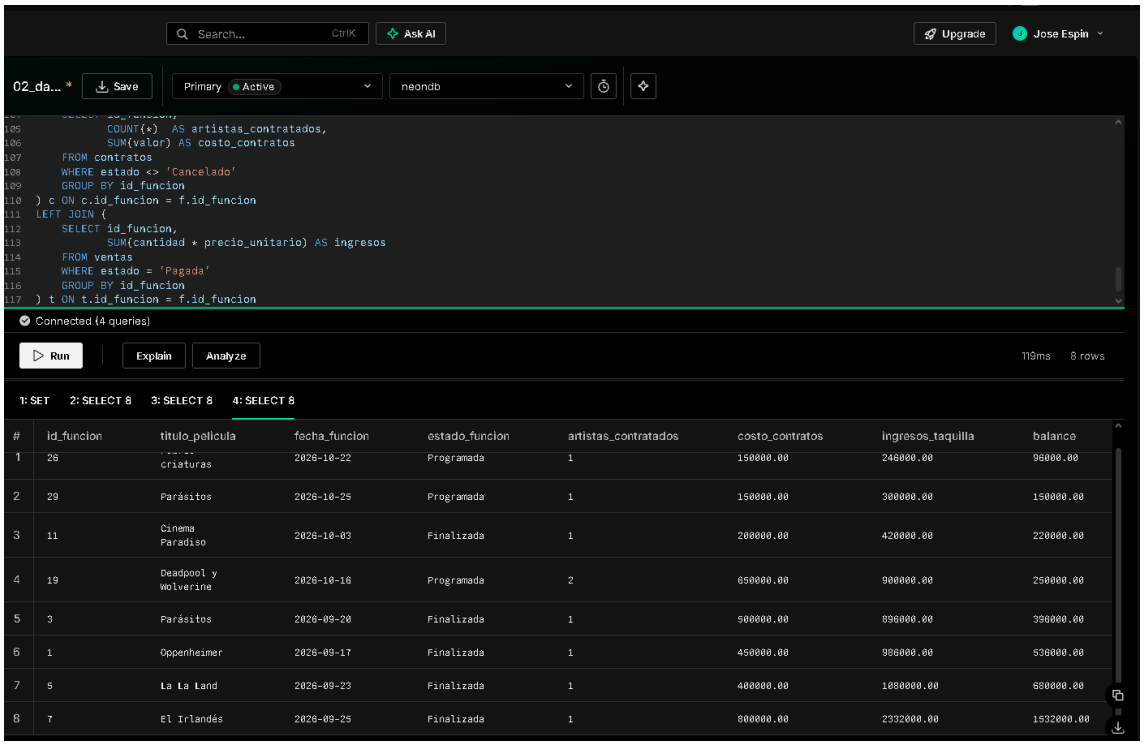


Fig. 12. Comparación entre los costos de los contratos y los ingresos de taquilla por función.

Conclusión

El núcleo transaccional de Altamira Cine cumple con los requerimientos del primer avance: las ocho entidades del modelo están implementadas en PostgreSQL con sus llaves, restricciones e índices, y cuentan con datos de prueba coherentes entre sí. La integridad del negocio no depende de una sola tabla, sino que se distribuye entre restricciones CHECK y UNIQUE, ocho triggers sobre seis tablas y tres procedimientos almacenados con manejo de excepciones. Las consultas muestran que la base de datos ya permite responder preguntas útiles para la operación, como qué funciones tienen baja ocupación, cuáles espectadores compran más y si la taquilla cubre el costo de los contratos con artistas. Para los siguientes avances queda pendiente incorporar una entidad Películas, que evitaría repetir sus datos en cada función.

Repositorio del proyecto: https://github.com/tef333/Proyecto-integrador-cinema